Binary Search Patterns – Interview Cheat Sheet

1. GLOBAL / STRUCTURE-BASED BINARY SEARCH

Use this when the array has a GLOBAL ordering or structure. The decision of which half to discard is made using a boundary element (left, right, or target).

Key Comparison: Compare mid with boundary (nums[right], nums[left], or target).

General Template:

```
while (left < right) { mid = left + (right - left) / 2; if (condition using
boundary) { left = mid + 1; } else { right = mid; } }
```

Problems that use GLOBAL Binary Search:

- Search in Sorted Array
- Search Insert Position
- Find Minimum in Rotated Sorted Array
- Search in Rotated Sorted Array
- First & Last Position of Element
- Lower Bound / Upper Bound

2. LOCAL / SLOPE-BASED BINARY SEARCH

Use this when the array is NOT sorted but has a LOCAL property like a peak, mountain, or local minima. The decision depends only on neighboring elements.

Key Comparison: Compare mid with its neighbor (nums[mid+1] or nums[mid-1]).

General Template:

```
while (left < right) { mid = left + (right - left) / 2; if (nums[mid] > nums[mid +
1]) { right = mid; } else { left = mid + 1; } }
```

Problems that use LOCAL Binary Search:

- Find Peak Element
- Peak Index in Mountain Array
- Find Local Minima
- Bitonic Array Peak

3. HOW TO DECIDE WHICH BINARY SEARCH TO USE

Ask this single question before coding:

Am I comparing using a GLOBAL boundary or a LOCAL neighbor?

- If comparison is with target / nums[left] / nums[right] → GLOBAL Binary Search
- If comparison is with nums[mid+1] or nums[mid-1] → LOCAL Binary Search

One-line Rule: Global order → boundary comparison. Local slope → neighbor comparison.